

Image Enhancer

The Complete Lifecycle Guide

Dockerfile -> Image -> Docker Hub -> Pull | Frontend + Backend + DB | A Real Domain

Module 5: Docker Deep Dive | DevOps with Gen AI

What's New in This Version

The project now touches every stage of the real Docker lifecycle, not just local build-and-run:

Part	Covers
A	The full image lifecycle: Dockerfile -> build -> tag -> push to Docker Hub -> pull fresh, as if on a new machine
B	A genuine 3-tier stack: frontend + backend + a real PostgreSQL database, with proven persistence
C	Mapping a real domain: EC2, Route 53, Certbot HTTPS, and a production Compose override file

What Changed in the Code

- backend/server.js now connects to PostgreSQL (via the pg library) and records every enhancement's settings
- docker-compose.yml gained a third service: db (PostgreSQL), with its own named volume and a healthcheck
- backend now waits for the database to be genuinely READY, not just started, using depends_on: condition: service_healthy
- New files: docker-compose.prod.yml, frontend/nginx.prod.conf.template, frontend/nginx.bootstrap.conf.template

Time Estimate

Part A: 20 min. Part B: 20 min. Part C: 45-60 min (includes real EC2 + DNS propagation wait time).

PART A — THE FULL IMAGE LIFECYCLE: DOCKERFILE TO HUB TO PULL

STEP 1 Build Both Images With Real Version Tags

```
cd image_enhancer
docker build -t <your-dockerhub-username>/image-enhancer-backend:1.0.0 ./backend
docker build -t <your-dockerhub-username>/image-enhancer-frontend:1.0.0 ./frontend
```

Using your Docker Hub username as the prefix now saves a separate docker tag step later.

STEP 2 Log In and Push Both Images

```
docker login

docker push <your-dockerhub-username>/image-enhancer-backend:1.0.0
docker push <your-dockerhub-username>/image-enhancer-frontend:1.0.0
```

Visit hub.docker.com and confirm both repositories now exist under your account.

Tip: While you're here, also tag and push :latest for each, exactly like the tagging discipline from the previous Docker class - two docker tag commands, two more docker push commands.

STEP 3 Simulate a Completely Fresh Machine

Remove every local copy - images, containers, everything this project has ever built:

```
docker compose down
docker rmi <your-dockerhub-username>/image-enhancer-backend:1.0.0
docker rmi <your-dockerhub-username>/image-enhancer-frontend:1.0.0
docker image prune -f

docker images | grep image-enhancer
# Should return NOTHING
```

STEP 4 Pull Both Images Back Down and Run Purely From the Registry

Create a second compose file that uses image: instead of build: - this is exactly what you'd run on a real server that never has your source code, only Docker Hub access:

```
# docker-compose.pulled.yml
services:
  backend:
    image: <your-dockerhub-username>/image-enhancer-backend:1.0.0
    environment:
      - DB_HOST=db
      - DB_NAME=image_enhancer
      - DB_USER=app_user
      - DB_PASSWORD=app_password
    volumes:
      - enhanced-images:/app/uploads
    networks: [enhancer-net]
    depends_on:
      db:
        condition: service_healthy

  frontend:
    image: <your-dockerhub-username>/image-enhancer-frontend:1.0.0
    ports: ["8080:80"]
    networks: [enhancer-net]

  db:
    image: postgres:16-alpine
    environment:
      - POSTGRES_DB=image_enhancer
      - POSTGRES_USER=app_user
      - POSTGRES_PASSWORD=app_password
    volumes: [db-data:/var/lib/postgresql/data]
    networks: [enhancer-net]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app_user -d image_enhancer"]

networks:
  enhancer-net:
volumes:
```

```
enhanced-images:  
db-data:  
  
docker compose -f docker-compose.pulled.yml up -d  
# Docker will PULL, not build - watch the log output confirm this  
curl http://localhost:8080
```

Tip: This is the entire point of Docker Hub proven with your own hands: the app runs identically, having never touched your source code on this "fresh machine" at all.

PART B — FRONTEND + BACKEND + DATABASE, VERIFIED

STEP 1 Bring Up the Full 3-Tier Stack

```
docker compose up --build
```

Watch the startup order in the logs: db starts first, its healthcheck runs `pg_isready` repeatedly, and backend genuinely WAITS until that check passes before starting - confirm this by watching the timestamps.

STEP 2 Confirm the Database Connection From the API Itself

```
curl http://localhost:8080/api/health  
# {"status":"ok", ..., "db":"connected"}
```

STEP 3 Enhance an Image, Then Verify It Landed in Postgres - Not Just on Disk

Upload a photo through the browser, then query the database DIRECTLY, bypassing the API entirely:

```
docker compose exec db psql -U app_user -d image_enhancer -c "SELECT filename, width, height,  
brightness, saturation FROM enhancements ORDER BY id DESC LIMIT 5;"
```

You should see a real row matching what you just uploaded - proof the backend's INSERT genuinely reached the database, not just the filesystem.

STEP 4 Prove the Database Survives a Restart

```
docker compose restart backend db
curl http://localhost:8080/api/enhancements
# Your earlier enhancement should still be listed
```

STEP 5 Prove depends_on: condition: service_healthy Actually Matters

Temporarily remove the healthcheck: block from db in docker-compose.yml (comment it out), then:

```
docker compose up --build --force-recreate
```

Watch out: Without the healthcheck, backend may start BEFORE Postgres is actually ready to accept connections, and its first few DB queries can fail - watch docker compose logs backend for connection errors right at startup. Put the healthcheck back afterward; this step is purely to see the failure mode with your own eyes.

PART C — MAPPING A REAL DOMAIN

This section assumes an EC2 instance and a Route 53 hosted zone already exist (Classes 9, 14). We'll deploy this exact multi-container stack to it, with real HTTPS.

STEP 1 Prepare the EC2 Instance

```
ssh -i your-key.pem ubuntu@<EC2_PUBLIC_IP>

sudo apt update
sudo apt install -y docker.io docker-compose-plugin certbot
sudo usermod -aG docker ubuntu
# log out and back in for the group change to apply
```

STEP 2 Copy the Project to the Server and Point DNS at It

```
# From your own machine:
scp -i your-key.pem -r image_enhancer ubuntu@<EC2_PUBLIC_IP>:~/
```

In Route 53, create an A record for your subdomain (e.g. images.friendscomputer.store) pointing at the EC2 instance's Elastic IP - exactly the pattern from the earlier Route 53 class.

Tip: Allow a few minutes for DNS to propagate before continuing - confirm with `dig images.yourdomain.com` or `nslookup`.

STEP 3 Deploy With the BOOTSTRAP Config First (No HTTPS Yet)

You cannot request a certificate before Nginx is running, and the FULL production config references a certificate that doesn't exist yet - so start with the HTTP-only bootstrap config.

```
cd ~/image_enhancer/frontend
cp nginx.bootstrap.conf.template nginx.prod.conf
sed -i 's/YOUR_DOMAIN/images.yourdomain.com/g' nginx.prod.conf
cd ~/image_enhancer

docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build

curl -I http://images.yourdomain.com
# Expect: HTTP/1.1 200 OK (plain HTTP, working, no certificate yet)
```

STEP 4 Obtain the Real Certificate

```
sudo mkdir -p /var/www/certbot
sudo certbot certonly --webroot \
  -w /var/www/certbot \
  -d images.yourdomain.com \
  --email you@example.com --agree-tos --no-eff-email
```

Certbot writes the certificate to /etc/letsencrypt/live/images.yourdomain.com/ - which docker-compose.prod.yml already bind-mounts into the frontend container.

STEP 5 Switch to the Full HTTPS Config

```
cd ~/image_enhancer/frontend
cp nginx.prod.conf.template nginx.prod.conf
sed -i 's/YOUR_DOMAIN/images.yourdomain.com/g' nginx.prod.conf
cd ~/image_enhancer
docker compose -f docker-compose.yml -f docker-compose.prod.yml restart frontend
```

STEP 6 Verify HTTPS Works End to End

```
curl -I https://images.yourdomain.com
# Expect: HTTP/2 200
```

Open https://images.yourdomain.com in a browser - confirm the padlock icon shows a valid, trusted certificate, then upload a real image and confirm it enhances correctly, exactly as it did locally.

STEP 7 Set Up Auto-Renewal

```
sudo certbot renew --dry-run
```

Let's Encrypt certificates expire every 90 days. A dry run confirms renewal will work; add a cron job or systemd timer running certbot renew periodically so this never needs to be done by hand again.

Full Verification Checklist

Check	Done?
Both images built, pushed, and confirmed visible on hub.docker.com	☐
All local copies removed, then successfully pulled fresh and run from the registry alone	☐
docker compose up shows backend waiting for db's healthcheck before starting	☐
/api/health reports db: connected	☐
A real row appears in Postgres after uploading an image (verified via psql directly)	☐
Enhancement data survives a restart of both backend and db	☐
EC2 instance has Docker, the Compose plugin, and Certbot installed	☐
Route 53 A record resolves to the EC2 instance's IP	☐
Bootstrap HTTP config deployed and confirmed working before requesting a certificate	☐
Certificate obtained successfully via Certbot webroot mode	☐
Production HTTPS config deployed, browser shows a trusted padlock	☐
certbot renew --dry-run succeeds	☐

Troubleshooting Quick Reference

Symptom	Likely Cause & Fix
docker push rejected: "denied"	Image tag doesn't start with your exact Docker Hub username, or docker login wasn't run
Pulled stack can't reach the db service	Confirm docker-compose.pulled.yml's network name and service names match exactly - typos here silently break service discovery
Backend crashes on startup with a DB connection error	The healthcheck: block is missing or misconfigured - re-check Part B, Step 5
Certbot fails: "Connection refused" during the challenge	Port 80 isn't reachable from the internet - check the EC2 Security Group allows inbound 80/443, and the bootstrap config is actually running
Certbot fails: "Invalid response"	DNS hasn't propagated yet, or the A record points at the wrong IP - verify with dig before retrying
Browser shows a certificate warning	Confirm nginx.prod.conf's server_name and the certificate's domain match EXACTLY, and that you restarted frontend after switching configs

Reflection prompt for class discussion: *why does the bootstrap config need to exist as a SEPARATE file at all - why not just start with the full HTTPS config and let it fail once, then fix it? Walk through exactly what happens on container startup with a missing certificate file.*